

# **ARTIFICIAL INTELLIGENCE**

## **Assignment- 1**

### **AI-Based Smart Water Tank Monitoring System**

**GROUP-14**

**Submitted by:**

Name : JINCE JOSE

Class : S7, CSE-A

Roll No. : 33

# 1. Introduction

Managing water tanks manually often results in overflow, insufficient water storage, and unnecessary energy usage. This AI-based smart monitoring system uses an intelligent software agent to continuously observe the water level and automatically operate the pump. The system ensures efficient water utilization, minimizes human effort, and improves the reliability of household water management..

## 2. PEAS Table

| Component           | Modified Description                                                                                                                                                |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Performance Measure | Maintain an optimal water level, eliminate overflow and water shortages, minimize unnecessary pump operations, and improve water and energy efficiency.             |
| Environment         | A virtual water tank environment that simulates water inflow, household consumption patterns, and changing water source conditions for intelligent decision-making. |
| Actuators           | A software-controlled virtual pump that automatically starts or stops based on the intelligent agent's decisions to regulate the tank's water level.                |
| Sensors             | A simulated water level sensor that continuously detects and reports the tank's water level, including realistic measurement variations.                            |

## 3. Environment properties

| Property         | Classification       | Modified Reason                                                                                                                                          |
|------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Observability    | Partially Observable | The agent depends on sensor readings that may include minor estimation errors, so the complete state of the tank cannot always be determined accurately. |
| Number of Agents | Single Agent         | A single intelligent agent performs all monitoring and pump control operations without assistance from other agents.                                     |
| Determinism      | Stochastic           | Changes in household water demand and water supply occur unpredictably, creating uncertainty in the environment.                                         |

|                              |                   |                                                                                                                                 |
|------------------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Episodic / Sequential</b> | <b>Sequential</b> | The agent's present action directly influences future water levels and subsequent decisions.                                    |
| <b>Static / Dynamic</b>      | <b>Dynamic</b>    | The environment changes continuously as water is consumed and refilled, requiring constant monitoring by the agent.             |
| <b>Discrete / Continuous</b> | <b>Continuous</b> | Water level varies smoothly over time, while the agent periodically measures and responds to these changes.                     |
| <b>Known / Unknown</b>       | <b>Known</b>      | The agent operates according to predefined simulation rules, pump behavior, and threshold values, making the environment known. |

## 4. Rationality Measure

The intelligent agent continuously monitors the tank's water level and selects the most appropriate action to maintain efficient operation. When the water level drops below **25%**, the agent automatically switches the pump **ON** to refill the tank. Once the level reaches **95%**, it turns the pump **OFF** to prevent overflow. By maintaining separate ON and OFF threshold values, the system reduces unnecessary pump switching, conserves electrical energy, and increases the operational life of the pump while ensuring a steady water supply.

## 5. Innovation Point

### 1. Smart Mobile Alerts

The system sends instant notifications to the user's mobile device whenever the water level becomes critically low, the tank is full, or an abnormal event is detected. This allows users to monitor the system remotely.

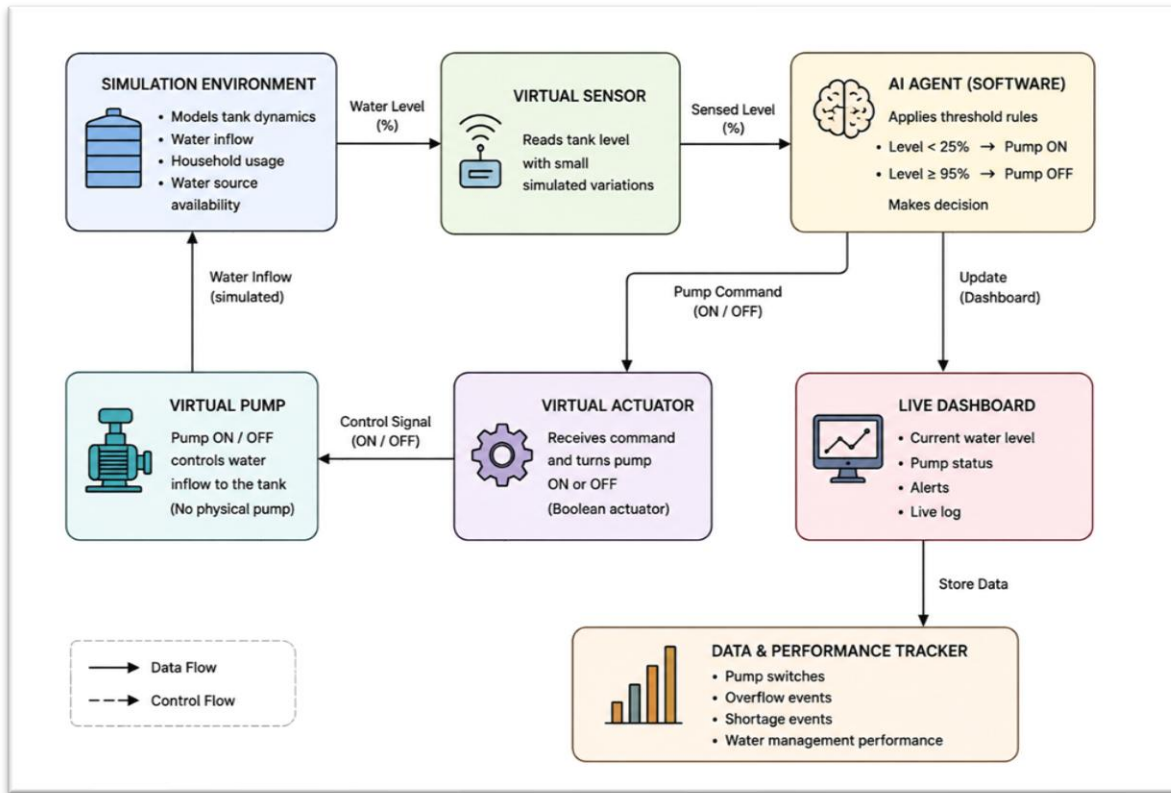
### 2. Water Quality Monitoring

The system can be integrated with water quality sensors to monitor parameters such as pH, turbidity, or contamination levels. If poor water quality is detected, the user is immediately notified to ensure safe water usage.

### 3. Solar-Powered Pump Optimization

The intelligent agent can prioritize operating the water pump when solar energy is available. This reduces dependence on grid electricity, lowers operating costs, and promotes environmentally friendly water management.

## 6. Diagram



## 7. Conclusion

The AI-Based Smart Water Tank Monitoring System shows how an intelligent software agent can automate water level management in a simulated environment. By monitoring the tank continuously and controlling the pump automatically, the system prevents overflow and water shortages while reducing electricity consumption. This approach improves water management efficiency and can be easily extended with IoT sensors, cloud connectivity, and mobile applications for real-world smart water management.

## 8. Simulation

The simulation illustrates the working of the AI-Based Smart Water Tank Monitoring System in a virtual environment. The intelligent agent continuously receives water level data from the simulated sensor and compares it with predefined threshold values. Based on these readings, the agent automatically turns the virtual pump **ON** when the water level is low and **OFF** when the tank reaches the desired level. During execution, the dashboard displays real-time information such as the current water level, pump status, and system activity. After the simulation is completed, the system provides a performance report that includes the total number of pump operations, overflow incidents, shortage incidents, and the overall efficiency of water management.

## 9a. Console Simulation — smart\_water\_tank.java

```
SmartWaterTank.java - smart-water-tank

1  /**
2   * AI-Based Smart Water Tank Monitoring System
3   * -----
4   * A simple-reflex / model-based rational agent that monitors the water level
5   * of an overhead tank (via an ultrasonic level sensor) and controls a motor
6   * pump to prevent both OVERFLOW and DRY-RUN (shortage), while also reacting
7   * to stochastic household water consumption and occasional sensor noise.
8   *
9   * Compile : javac SmartWaterTank.java
10  * Run      : java SmartWaterTank
11  */
12
13  import java.util.ArrayList;
14  import java.util.List;
15  import java.util.Random;
16
17  public class SmartWaterTank {
18
19      // ----- CONFIGURATION -----
20      static final double TANK_CAPACITY_L = 1000.0; // litres, full tank capacity
21      static final double LOW_THRESHOLD = 25.0; // % → pump turns ON at/below this level
22      static final double HIGH_THRESHOLD = 95.0; // % → pump turns OFF at/above this level
23      static final double PUMP_FLOW_RATE_LPM = 40.0; // litres pumped per minute (simulated tick)
24      static final double SENSOR_NOISE_STD = 1.5; // % standard deviation of sensor noise
25      static final int SIMULATION_TICKS = 40; // number of simulated time-steps (minutes)
26      static final double DRY_SOURCE_PROBABILITY = 0.03; // chance the water source runs dry this tick
27
28      // =====
29      // LOG ENTRY (replaces Python tuple in agent.log list)
30      // =====
31      static class LogEntry {
32          final int tick;
33          final double percept;
34          final boolean pumpOn;
35
36          LogEntry(int tick, double percept, boolean pumpOn) {
37              this.tick = tick;
38              this.percept = percept;
39              this.pumpOn = pumpOn;
40          }
41      }
42
43      // =====
44      // ENVIRONMENT
45      // =====
46      /**
47       * The environment: physical tank + stochastic household demand.
48       */
49      static class WaterTankEnvironment {
50
51          private final double capacityL;
52          private double levelL;
53          private boolean pumpOn;
54          private boolean sourceDry;
55          private final Random rng;
56
57          WaterTankEnvironment(double capacityL, double startPct, Random rng) {
58              this.capacityL = capacityL;
59              this.levelL = capacityL * startPct / 100.0;
60              this.pumpOn = false;
61              this.sourceDry = false;
62              this.rng = rng;
63          }
64
65          /** True tank level as a percentage [0..100]. */
66          double trueLevelPct() {
67              return Math.max(0.0, Math.min(100.0, 100.0 * levelL / capacityL));
68          }
69
70          /** Return a noisy percept of the tank level (simulates a real sensor). */
71          double sensorReading() {
72              double noise = gaussNoise(SENSOR_NOISE_STD);
73              double raw = trueLevelPct() + noise;
74              return Math.round(Math.max(0.0, Math.min(100.0, raw)) * 10.0) / 10.0;
75          }
76
77          /**
78           * Advance the environment by one tick given the agent's actuator command.
79           * @return consumption (litres) this tick
80           */
81          double step(boolean pumpCommand) {
82              // Random household consumption (stochastic demand): 5..25 litres this tick
83              double consumption = 5.0 + rng.nextDouble() * 20.0;
84              levelL -= consumption;
85
86              // Occasionally the borewell / municipal source runs dry
87              sourceDry = rng.nextDouble() < DRY_SOURCE_PROBABILITY;
88
89              pumpOn = pumpCommand;
90              if (pumpCommand && !sourceDry) {
91                  levelL += PUMP_FLOW_RATE_LPM;
92              }
93
94              // physical bounds
95              levelL = Math.max(0.0, Math.min(capacityL, levelL));
96              return consumption;
97          }
98      }
99  }
```

```

92     }
93
94     // physical bounds
95     levelL = Math.max(0.0, Math.min(capacityL, levelL));
96     return consumption;
97 }
98
99 boolean isSourceDry() { return sourceDry; }
100
101 /** Box-Muller transform: Gaussian noise with mean=0 and given std. */
102 private double gaussNoise(double std) {
103     double u, v;
104     do { u = rng.nextDouble(); } while (u == 0.0);
105     do { v = rng.nextDouble(); } while (v == 0.0);
106     return std * Math.sqrt(-2.0 * Math.log(u)) * Math.cos(2.0 * Math.PI * v);
107 }
108 }
109
110 // =====
111 // AGENT
112 // =====
113 /**
114  * Model-based reflex agent with hysteresis control (a rational agent).
115  * - Percept:      noisy water-level reading from the sensor
116  * - Internal state: last known pump status
117  * - Action:       PUMP_ON / PUMP_OFF / NO_CHANGE
118  * - Rationality: maximises the performance measure (water availability,
119  * zero overflow, minimum pump switching, energy saved) given only the
120  * partial/noisy percepts it receives.
121  */
122 static class SmartTankAgent {
123
124     private boolean pumpOn = false;
125     private final List<LogEntry> log = new ArrayList<>();
126
127     // performance-measure counters
128     private int overflowEvents = 0;
129     private int shortageEvents = 0;
130     private int pumpSwitches = 0;
131
132     boolean act(double perceptPct, int tick) {
133         boolean previousState = pumpOn;
134
135         // Simple reflex rule-set with hysteresis to avoid rapid switching
136         if (perceptPct <= LOW_THRESHOLD) {
137             pumpOn = true;
138         } else if (perceptPct >= HIGH_THRESHOLD) {
139             pumpOn = false;
140         }
141         // else: keep previous state (model-based memory of last decision)
142
143         if (pumpOn != previousState) {
144             pumpSwitches++;
145         }
146
147         if (perceptPct >= 99.5) overflowEvents++;
148         if (perceptPct <= 2.0) shortageEvents++;
149
150         log.add(new LogEntry(tick, perceptPct, pumpOn));
151         return pumpOn;
152     }
153
154     int getOverflowEvents() { return overflowEvents; }
155     int getShortageEvents() { return shortageEvents; }
156     int getPumpSwitches()   { return pumpSwitches; }
157 }
158
159 // =====
160 // SIMULATION RUNNER
161 // =====
162 static void runSimulation(Random rng) {
163     WaterTankEnvironment env = new WaterTankEnvironment(TANK_CAPACITY_L, 60, rng);
164     SmartTankAgent agent = new SmartTankAgent();
165
166     // Header
167     System.out.printf("%4s | %12s | %10s | %6s | %9s\n",
168         "tick", "SensorLevel%", "TrueLevel%", "Pump", "SourceDry");
169     System.out.println("-".repeat(55));
170
171     for (int tick = 1; tick <= SIMULATION_TICKS; tick++) {
172         double percept = env.sensorReading();
173         boolean action = agent.act(percept, tick);
174         env.step(action);
175
176         System.out.printf("%4d | %12.1f | %10.1f | %6s | %9s\n",
177             tick,
178             percept,
179             env.trueLevelPct(),
180             action ? "ON" : "OFF",
181             env.isSourceDry() ? "YES" : "no");
182     }
183
184     System.out.println("-".repeat(55));
185     System.out.println("SIMULATION SUMMARY");
186     System.out.printf("  Final tank level      : %1f%%\n", env.trueLevelPct());
187     System.out.printf("  Pump ON/OFF switches   : %d\n", agent.getPumpSwitches());
188     System.out.printf("  Over-flow events       : %d\n", agent.getOverflowEvents());
189     System.out.printf("  Shortage events        : %d\n", agent.getShortageEvents());
190
191     double efficiency = 100.0
192         - (agent.getOverflowEvents() + agent.getShortageEvents()) * 100.0 / SIMULATION_TICKS;
193     System.out.printf("  Water-management score: %1f / 100%\n", efficiency);
194 }
195
196 // =====
197 // ENTRY POINT
198 // =====
199 public static void main(String[] args) {
200     Random rng = new Random(42L); // reproducible run for report/demo
201     runSimulation(rng);
202 }
203
204 }

```

## 9b. Software Console — smart\_water\_tank.html

```
smart_water_tank.html x

1  /* DESIGN TOKENS — Emerald + Deep Violet theme */
2  :root {
3    --bg:           #07040f;
4    --surface-1:    #100c1e;
5    --em:           #10b981; /* emerald primary */
6    --em-bright:    #34d399;
7    --vio:          #8b5cf6; /* violet secondary */
8    --vio-bright:   #a78bfa;
9    --rose:         #fb7185;
10   --orange:       #fb923c;
11   --text-1:       #eef2ff;
12   --text-2:       #9ca3af;
13 }
14
15 <!-- HEADER -->
16 <header>
17   <div class="brand">
18     <div class="brand-logo"><!-- gradient logo --> </div>
19     <div class="brand-text">
20       <h1>AquaMind Monitor</h1>
21       <p>AI Smart Water Tank · Java Edition</p>
22     </div>
23     <div class="status-badge" id="statusBadge">
24       <span class="pulse"></span>
25       <span id="statusText">Idle</span>
26     </div>
27   </header>
28
29 <!-- MAIN GRID -->
30 <div class="main-grid">
31   <!-- Tank visual, stats, log on left -->
32   <!-- Controls, PEAS, Score on right -->
33 </div>
34
35 // Agent (mirrors SmartTankAgent.act in Java)
36 function agentAct(percept) {
37   const prev = pumpOn;
38   if (percept <= LOW_TH) pumpOn = true;
39   else if (percept >= HIGH_TH) pumpOn = false;
40   // else: keep previous state (model-based memory)
41   if (pumpOn !== prev) switches++;
42   if (percept >= 99.5) overflow++;
43   if (percept <= 2) shortage++;
44 }
45
46 // Environment step
47 function envStep() {
48   const consumption = 5 + Math.random() * (demandMax - 5);
49   level -= consumption;
50   const sourceDry = Math.random() < dryProb;
51   if (pumpOn && !sourceDry) level += FLOW_RATE;
52   level = Math.max(0, Math.min(CAPACITY, level));
53   return sourceDry;
54 }
```



# 10.Output

## 10a. Console output (smart\_water\_tank.java)

```
SmartWaterTank.java — Terminal Output
> javac SmartWaterTank.java
> java SmartWaterTank
```

| Tick | SensorLevel% | TrueLevel% | Pump | SourceDry |
|------|--------------|------------|------|-----------|
| 1    | 59.5         | 58.9       | OFF  | no        |
| 2    | 60.0         | 57.6       | OFF  | no        |
| 3    | 58.0         | 55.3       | OFF  | no        |
| 4    | 54.4         | 54.5       | OFF  | no        |
| 5    | 55.7         | 53.6       | OFF  | no        |
| 6    | 52.6         | 51.9       | OFF  | no        |
| 7    | 49.5         | 49.8       | OFF  | no        |
| 8    | 49.7         | 48.4       | OFF  | no        |
| 9    | 48.6         | 46.6       | OFF  | no        |
| 10   | 44.6         | 45.2       | OFF  | no        |
| 11   | 46.0         | 44.4       | OFF  | no        |
| 12   | 44.0         | 43.5       | OFF  | no        |
| 13   | 42.7         | 42.1       | OFF  | no        |
| 14   | 41.6         | 40.5       | OFF  | no        |
| 15   | 40.8         | 38.8       | OFF  | no        |
| 16   | 39.8         | 37.9       | OFF  | no        |
| 17   | 36.4         | 36.7       | OFF  | no        |
| 18   | 37.8         | 35.3       | OFF  | YES       |
| 19   | 31.4         | 33.4       | OFF  | no        |
| 20   | 33.0         | 31.7       | OFF  | no        |
| 21   | 31.1         | 29.2       | OFF  | no        |
| 22   | 29.0         | 26.9       | OFF  | no        |
| 23   | 25.9         | 25.8       | OFF  | no        |
| 24   | 25.7         | 24.9       | OFF  | no        |
| 25   | 27.4         | 24.4       | OFF  | no        |
| 26   | 26.7         | 22.1       | OFF  | no        |
| 27   | 23.7         | 24.5       | ON   | no        |
| 28   | 23.9         | 26.9       | ON   | no        |
| 29   | 28.1         | 29.2       | ON   | no        |
| 30   | 29.5         | 32.1       | ON   | no        |
| 31   | 30.3         | 35.1       | ON   | no        |
| 32   | 34.1         | 37.6       | ON   | no        |
| 33   | 38.4         | 40.5       | ON   | no        |
| 34   | 39.3         | 42.4       | ON   | no        |
| 35   | 42.1         | 45.6       | ON   | no        |
| 36   | 45.9         | 47.4       | ON   | no        |
| 37   | 47.6         | 49.7       | ON   | no        |
| 38   | 46.9         | 52.1       | ON   | no        |
| 39   | 51.2         | 55.6       | ON   | no        |
| 40   | 57.1         | 57.3       | ON   | no        |

```
SIMULATION SUMMARY
Final tank level      : 57.3%
Pump ON/OFF switches : 1
Overflow events       : 0
Shortage events       : 0
Water-management score: 100.0 / 100
```



## 10b. Software console output (smart\_water\_tank.html)

